

Mobile Application Development

Lecture 05B

Today's Agenda

- Types of Layouts
 - Constraints Layout
 - Extending Layouts

Constraints Layout

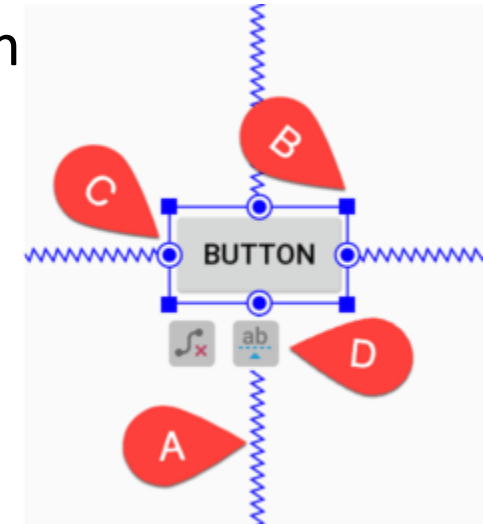
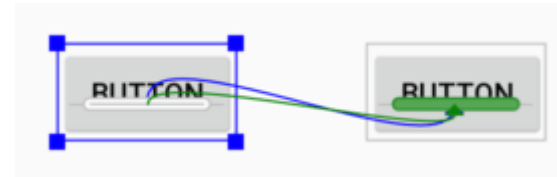
- Android ConstraintLayout is used to define a layout by assigning constraints for every child view/widget relative to other views present.
- The aim of ConstraintLayout is to improve the performance of the applications by removing the nested views with a flat and flexible design.
- A view inside the ConstraintLayout has handles(or anchor points) on each side which are used to assign the constraints.

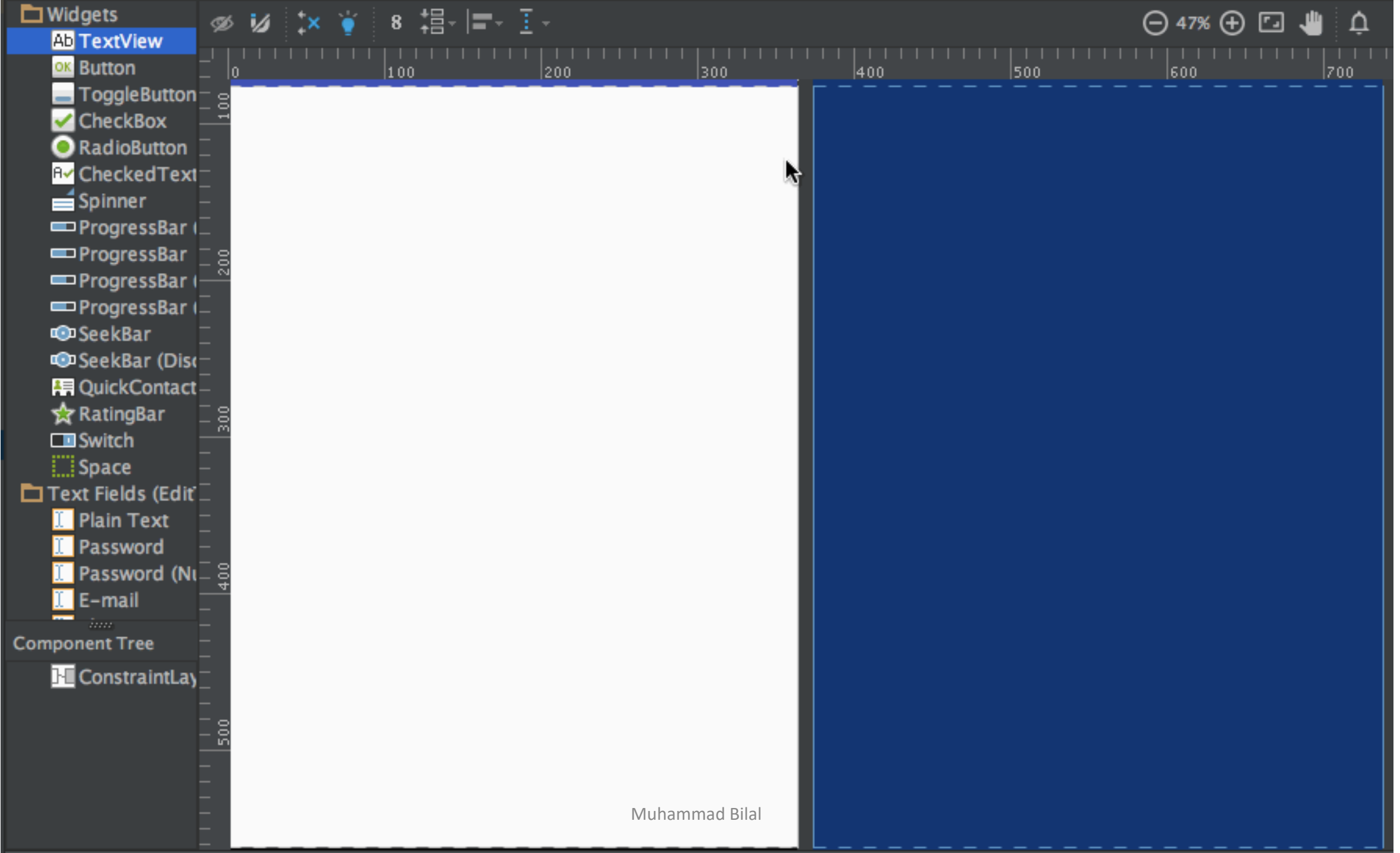
What is constraint

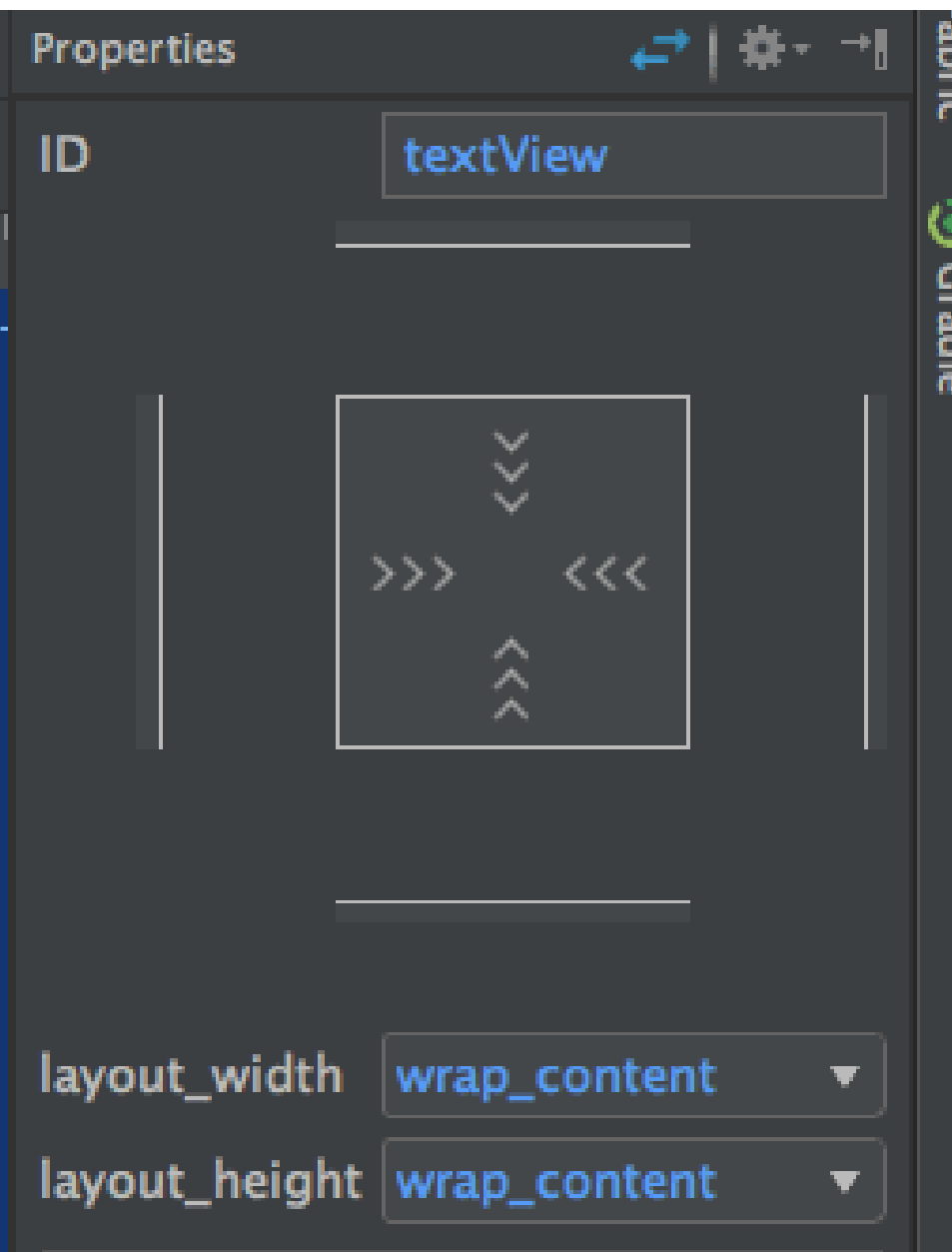
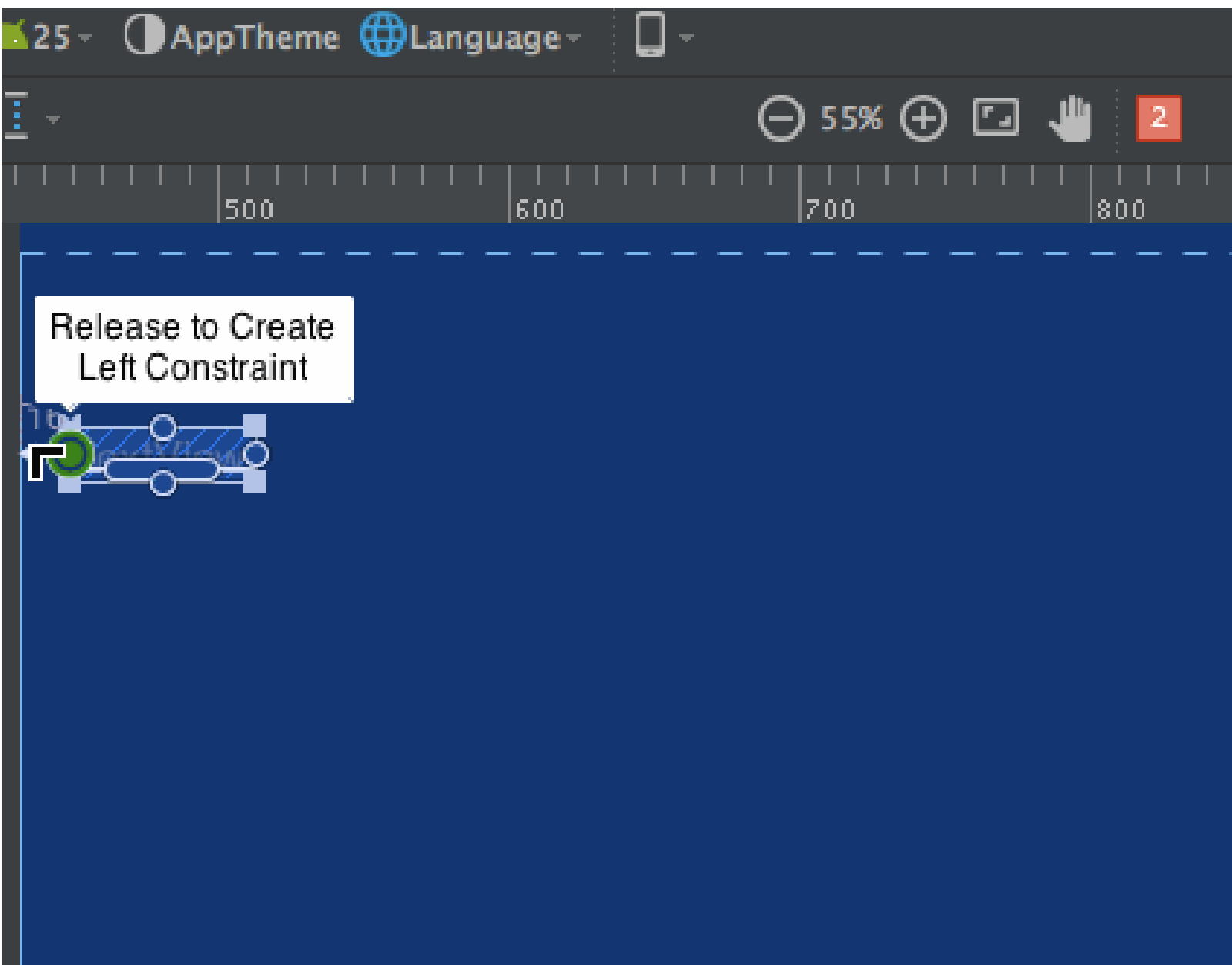
- The fundamental building block of constraint layout is creating constraints.
- A constraint defines a relationship between two widgets/views in the layout and controls how those widgets/views will be positioned within layout.

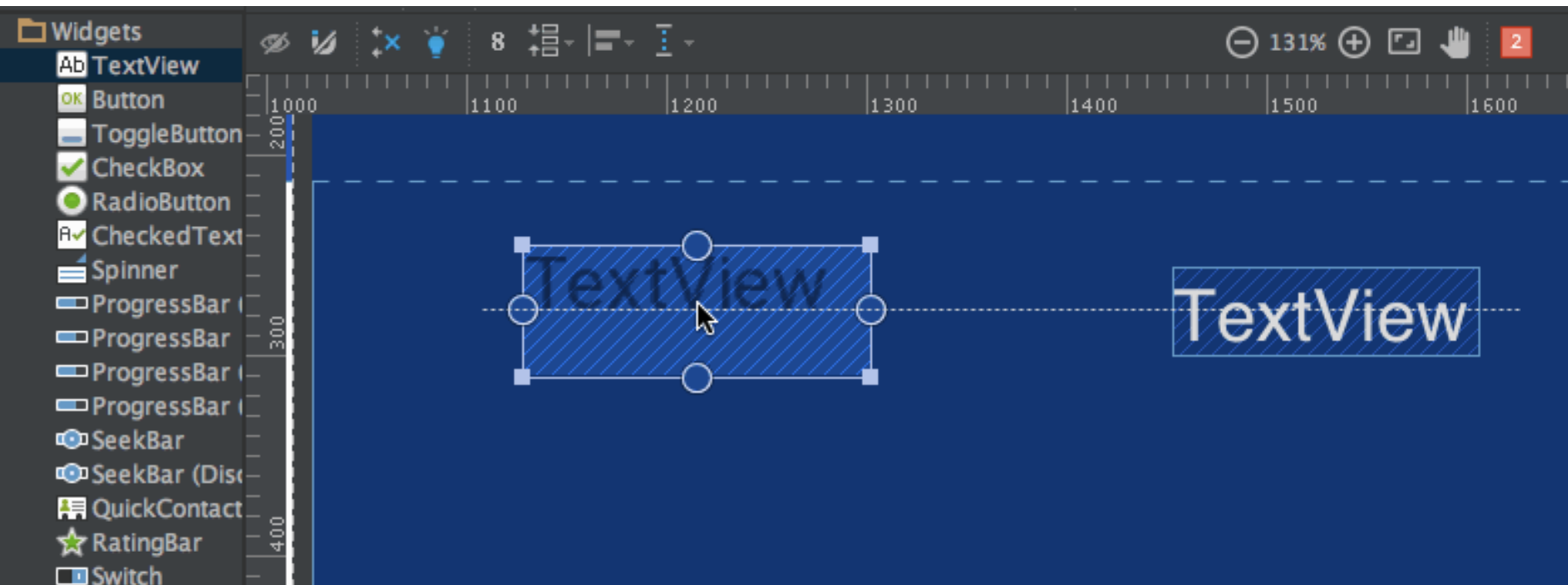
Constraints Layout

- Any View in constraints layout will have four types of handles:
 - The spring-like lines (A) represent established constraint connections leading from the sides of the widget to the targets.
 - **Resize handle** – It's presented on the four corners and is used to resize the view, but keeping its constraints intact. (B) represent resize handle.
 - **Side handle** – It's the circular handle (C) present on the center of each side. It's used to set the top, left, bottom and right constraints of the view.
 - **Baseline handle** – It's used to align the baseline with another textview in layout. It can be visible by clicking (D).
- Constraints are one way.



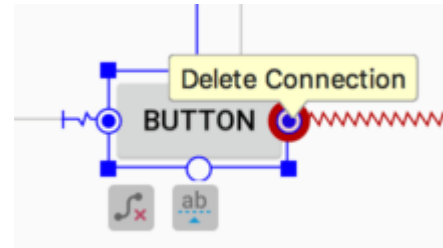




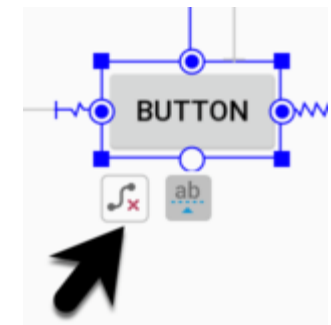


Deleting Constraints

- To delete an individual constraint, simply click within the anchor to which it is connected. The constraint will then be deleted from the layout (when hovering over the anchor it will glow red to indicate that clicking will perform a deletion):

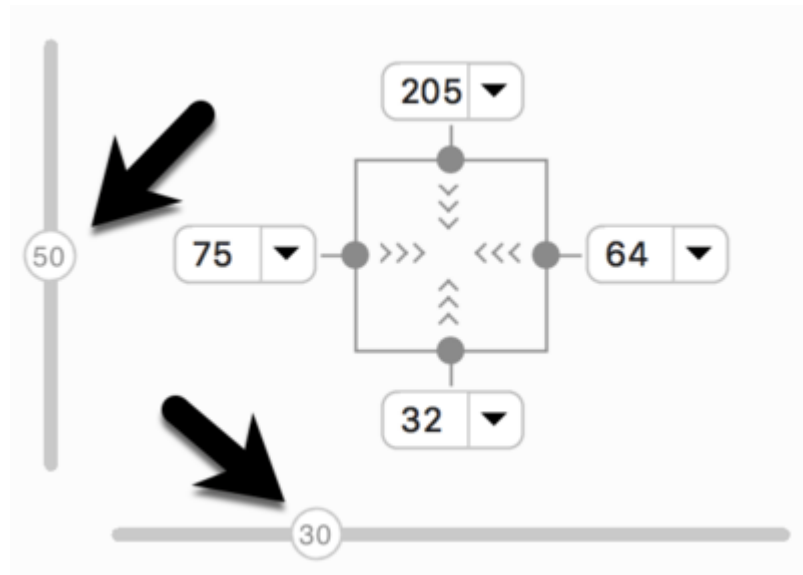


- Alternatively, remove all of the constraints on a widget by selecting it and clicking on the Delete All Constraints button which appears next to the widget when it is selected in the layout.



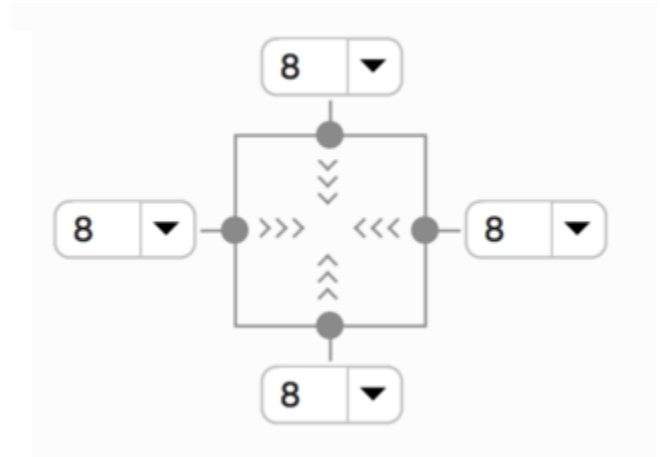
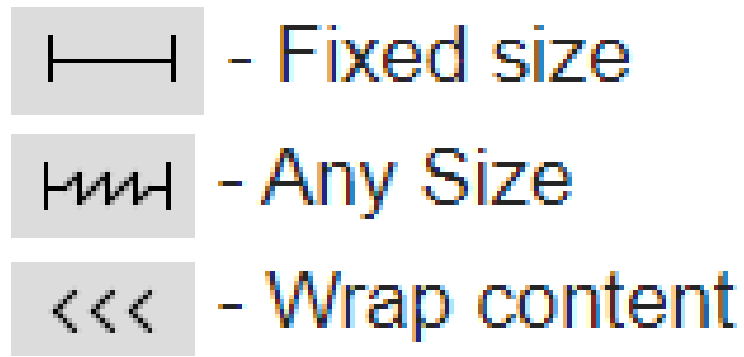
Adjusting Constraint Bias

- The two sliders indicated by the arrows in the figure are used to control the bias of the vertical and horizontal opposing constraints of the currently selected widget.



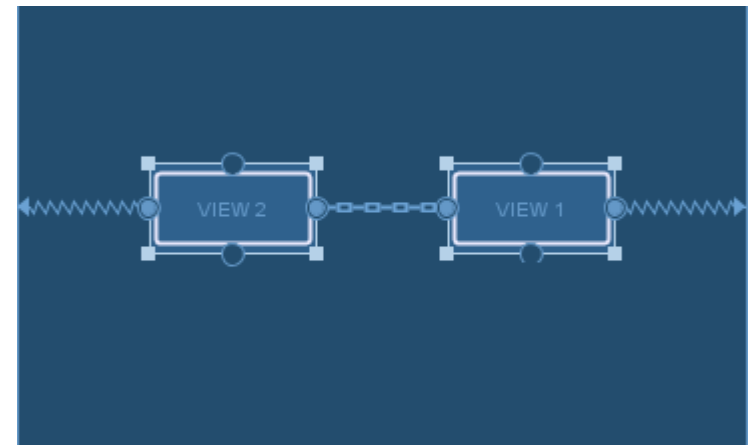
Configuring Widget Dimensions

- The inner dimensions of a widget within a ConstraintLayout can also be configured using the Inspector. Widget dimensions can be set to wrap content, fixed or match constraint modes. The prevailing settings for each dimension on the currently selected widget are shown within the square representing the widget in the Inspector as illustrated in Figure below:

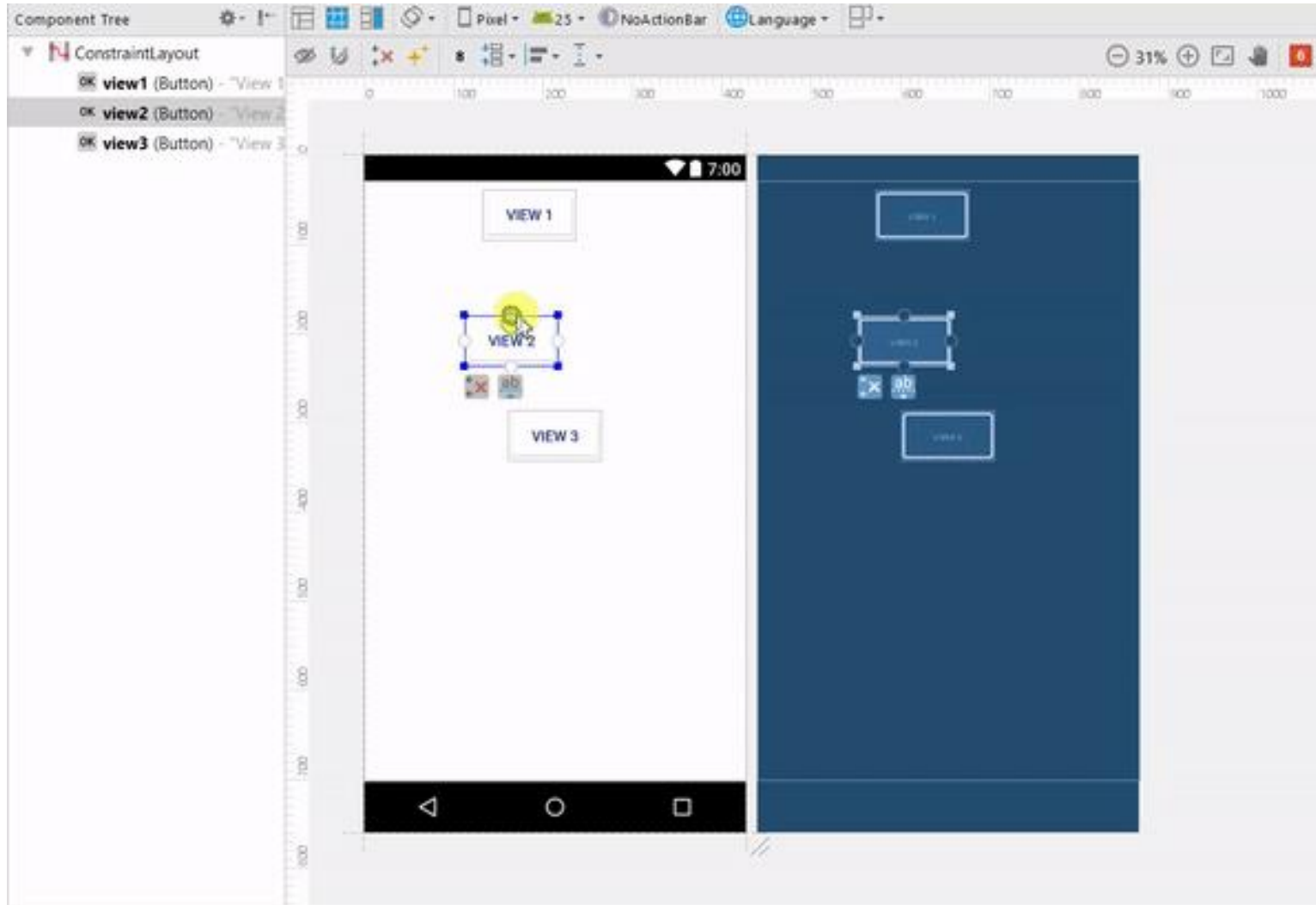


Constraint Layout Chains

- Chains are a specific kind of constraint which allow us to share space between the views within the chain and control how the available space is divided between them. A chain is a group of views that are linked to each other with bi-directional position constraints.
- Chains are two ways.
- For example, figure 2 shows two views that both have a constraint to each other, thus creating a horizontal chain.



Constraint Layout Chains



Extending Layout

- **Include:**
- When designing the Android layout file, we sometimes might need to re-use larger components that require a special layout. To efficiently achieve this, we can use the `<include/>` tag to embed another layout inside the current layout.

```
included_layout.xml

<?xml version="1.0" encoding="utf-8"?>
<layout xmlns:android="http://schemas.android.com/apk/res/android">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/world"/>
</layout>
```

Extending Layout

```
<layout xmlns:android="http://schemas.android.com/apk/res/android">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical">

        <TextView
            android:id="@+id/hello"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>

        <include
            android:id="@+id/world1"
            layout="@layout/included_layout"/>

        <include
            android:id="@+id/world2"
            layout="@layout/included_layout"/>

    </LinearLayout>
</layout>
```